Plain-English Project Method

Project: four-square-letterhead.ai · Project/contract edition: v1.8

FSL-v1.8-SKILL.md is the authoritative editable source for this method; PDF and DOCX are derived reading copies. This v1.8 method consolidates v1.7 and the agreed lean refinements. Framework adoption and publication are recorded separately; consuming projects adopt it explicitly.

1. Own the work; explain the promise

A skill, not a form

FSL is a compact project identity, contract and working method for humans and agents. Explain enough to act correctly, recognize boundaries and verify the outcome. Ordinary English can be the complete deliverable for a skill. Do not invent software, agents or inventories merely to fill the page.

Use one page when sufficient, two or three when necessary explanation would otherwise be lost; three is the letterhead maximum. Extra pages have no compulsory assignments. Remove repetition and empty forms before reducing type size. Detailed logs and inventories stay in project files; no YAML schema or compulsory iteration ledger is needed.

Four continuing responsibilities

Start with **[SPEC | SYSTEM | AI | MEMORY]**. SPEC and MEMORY are fixed; SYSTEM and AI are recommended defaults. Preserve justified alternatives such as TEMPLATES/SKILL or CODE/DOCS. These are responsibilities, not development phases. **Squares own artifacts; routes cross only the artifacts needed for the claimed outcome.**

SPEC starts with the project's completed FSL letterhead, then supporting scope, requirements, contracts, guardrails, acceptance and routes. Appendix, errata and amendments appear only when useful. SPEC owns the adopted promises and their lifecycle; FSL is not a fifth square.

SYSTEM owns the deliverable and its checks: code, runtime, data, interfaces, infrastructure or English instructions. **AI** owns AI-specific agents, prompts, skills, tools, automation and evaluations where relevant. **MEMORY** retains decisions, context, limitations, glossary, mnemonics, handover and evidence links. Declare inactive AI scope; do not invent capability.

Ownership follows purpose, not authorship. Agent-written product code belongs to SYSTEM. For a project whose product is a reusable skill, the skill is its primary deliverable, not automatically a third-square artifact. Shared material has one authoritative owner; generated copies and references do not create competing originals.

One authority for each promise

Keep project work in the owning repository. The FSL repository holds the framework and clearly labeled reference examples, not consuming projects. Show project/contract identity separately from the FSL template version on every page. The project's FSL is the entry point; supporting binding details need an explicit adopted source, scope and version.

MUST / **MUST NOT** bind; **SHALL** / **SHALL NOT** are aliases. **SHOULD** / **SHOULD NOT** give strong guidance; **MAY** permits. Article, law, contract, policy and guardrail labels do not determine force. In rendered pages, normative words use capitals, bold and underline, not decorative emphasis. Give material binding promises stable clause IDs with baseline context; do not recycle IDs for unrelated obligations. Binding intent **MUST NOT** be silently weakened or bypassed. Skills and MEMORY cannot rewrite it; surface ambiguity for review.

Plain-English Project Method

Project: four-square-letterhead.ai · Project/contract edition: v1.8

2. Define the route; develop iteratively

Route, iteration, run and evidence

A **route** identifies a meaningful input-to-outcome path. An **iteration** is one focused loop of work, checking and learning. A **run** is an actual execution or review of a particular route. **Evidence** records what that run demonstrated. One iteration can include several runs and revisit the same route.

Use **R-01** for the primary route and add only other critical routes. Remove compulsory iteration numbering, not iterative work; teams may number loops internally. R-01 is not an iteration counter, run number or success badge. Keep its definition stable within its baseline; review changes rather than deleting touchpoints to fit a passing subset.

R-01: input → required touchpoints → observable output. State the expected outcome, exclusions, related clause IDs and contract/route baseline. **Route defines expected coverage; run records observed coverage.** An arrow line or sentence is enough; no touchpoint table is needed. Name material midpoint, conditional-path or asynchronous boundaries. Touchpoints are not repository branches.

Four recommended first E2E learning goals

For new projects, development **SHOULD** begin with four E2E learning goals: first establish the smallest complete path; then make it useful with representative behavior and data; then check an important failure, permission or recovery boundary; finally rehearse intended use in a representative release environment. Review each connected outcome before increasing scope.

These are the recommended first four learning goals, not four compulsory iterations, four routes or release approval. Repeat or split a goal as needed; explain a material departure instead of silently dropping it. Several loops may use the same route. Grow from the latest proven baseline; preserve previously proven behavior unless reviewed requirements change it. **Tiny complete path → useful behavior → boundary/recovery → intended-use rehearsal → grow.**

Existing systems enter from the present

Study or reverse-engineer as needed. Identify the reviewed repository/commit or artifact baseline, scope and known gaps. Separate observed behavior from adopted requirements; an old bug does not become law because it exists. Define the important routes and make only the claims their evidence supports.

Historical and future iteration plans are optional; existing projects need not recreate the four early goals. Do not rebuild working capability or rewrite history for adoption. A reviewed baseline may retain explicitly unproven claims. Conformance concerns the adopted scope and evidence, not the development method used throughout the project's past.

Show the full path on each run

Before each route run, show every declared touchpoint. Afterwards retain PASS, FAIL or NOT RUN for each, including unreached points, in an ordinary log or prose record. Run results are not conformance states. Record run identity, implementation revision, effective SPEC/route baseline, environment, substitutions, expected/observed outcome and evidence. Never prefill or silently inherit a previous PASS. Verify the connections and final outcome; component passes **MUST NOT** be presented as integrated E2E proof.

Exercise unchanged components without editing them. Stubs prove only their disclosed boundary; include AI only when the runtime claim depends on it. E2M goes to a midpoint; M2E goes from it. Both stay partial until meet-in-the-middle integration is checked. Parallel E2E paths keep separate results and check shared contracts/state. Negative routes can PASS by demonstrating expected rejection. Use safe representative environments, not implied destructive production tests.

Plain-English Project Method

3. Learn, preserve and release honestly

English can describe real work

For an English skill: **brief → source review → interpretation → draft → review → delivered document**. Expected: fidelity to sources and visible unresolved intent. Apply the skill to representative cases; written instructions alone are not universal proof. Checklist-based work is recommended. Agents **MAY** use Discover → Plan → Execute → Observe → Verify → Record under defined responsibilities. For background work in scope, describe job identity, progress, cancellation and final receipt.

Use analogies when they prevent confusion: the route is a map; the run is a journey. Structural correspondences, or homologies, can show how software and knowledge work both transform input into a checked outcome without requiring identical components. Explain where the comparison stops. Analogies and examples clarify adopted meaning; they neither create law nor substitute for evidence.

Use references without importing hidden rules

Name only essential technologies, libraries, standards or prior work, with purpose and canonical source. Pin a version or commit when meaning depends on it. REUSE uses capability; INHERIT explicitly adopts an identified contract; REFERENCE consults without adopting its law. A binding technology choice needs an explicit project clause, not merely a link or category label. Reuse proven capability before rebuilding its equivalent where appropriate.

Maintain the specification without rewriting it

Review draft intent directly before release. An appendix may grow during discovery or development. Preserve the released SPEC: appendix explains; errata corrects without changing intent; an adopted amendment changes intent. These are optional documents, not mandatory lifecycle stages or empty folders.

A proposal is not approval. Adoption identifies the authorized decision, affected baseline/clauses, exact change, effective scope/date and immutable record. Identify the applicable adopted changes and resolve overlaps before conformance claims. Reassess affected implementation and routes; changed intent is not proof of changed behavior. The next release should incorporate applicable adopted corrections and amendments into a new baseline, retaining the earlier record.

Preserve lineage; review the combination

For a new repository, main1 identifies the original first commit; initial lanes begin there and later versions continue corresponding predecessors. For an existing repository, adoption lanes can begin at the reviewed current baseline. Do not move main1 or fabricate old branches. Branch suffixes follow project releases, not template versions. Keep existing justified specializations.

Each lane stays understandable and self-validating within its declared scope and dependencies. A release records all four reviewed commits and their integration. Check that combination against the effective contract and critical routes, not four unrelated green badges. Unchanged squares need review, not ceremonial edits. Git records the snapshot; a branch name is not a frozen snapshot. An ordinary Markdown note suffices. **One version, four squares, all green.**

Make claims no stronger than their evidence

Review SPEC stability, verified primary deliverable, aligned second concern or declared inactivity, and understandable MEMORY. Scope proof as **CLAIM / SCOPE / ENVIRONMENT / EVIDENCE / RESULT**. Known binding violations are **NON-CONFORMANT**; missing evidence or unresolved intent is **NOT YET PROVEN**. **CONFORMANT** needs applicable evidence without unresolved violations. **CONFORMANT WITH EXCEPTION** needs authorized scope, clause, reason, evidence and review/expiry as applicable. A waiver cannot turn an unrun check into PASS.

Distinguish drafted, reviewed, tested, accepted and published work. A local package is not a published release; a public branch release is not a GitHub Releases-page entry. A released framework does not certify its consuming projects. MEMORY may remember law; MEMORY cannot make law.